

X アルゴリズム解説セッション

生成日: 2026-05-16 会話数: 78 ターン

▶ よーん

<https://github.com/xai-org/x-algorithm.git> を clone して

◀ Claude

/root/x-algorithm にクローンしました。主なディレクトリは
candidate-pipeline ・ grox ・ home-mixer ・ phoenix ・ thunder です。何か見たいものはありますか？

▶ よーん

そのリポジトリの情報だけを参考にして僕の質問に答えて。web search は禁止。* X の for you アルゴリズムでは、1日にアクション（ポスト、リプライ、リポスト、引用リポスト、ブックマーク、いいね）をすればするほど、ポストが伸びにくくなる？

◀ Claude

リポジトリを調べます。

◀ Claude

リポジトリの調査結果をまとめます。

結論：「アクションが多いほど伸びにくくなる」という仕組みはコードベース上に存在しない。

以下が根拠です。

アクションが「多いほど良い」方向に働く仕組み

[phoenix/run_retrieval.py](#) / [run_ranker.py](#)

ユーザーの直近32件のアクション履歴を Grok Transformer
に入力する。履歴が豊富なほどモデルの推薦精度が上がる。

[home-mixer/sources/phoenix_source.rs](#)

アクション履歴数が閾値（`PhoenixRetrievalNewUserHistoryThreshold`）未満だと「新規ユーザー用の別クラス
タ」にルーティングされる。つまりアクションが少ない方が「通常モデル」から外れる。

伸びを下げる仕組みとして存在するもの（アクション頻度とは無関係）

仕組み 対象 ファイル

****Author Diversity Dampening**** 同一著者が1フィードに複数回登場する場合に減衰
`author\_diversity\_scorer.rs`

****Spam Detection**** フォロワー数が少ないアカウントのリプライスパム `task\_spam\_detection.py`

****ネガティブアクション**** ブロック・ミュート・通報を受けたコンテンツのスコアが下がる
`weighted\_scorer.rs`

追跡されているアクション19種

favorite_score・**reply_score**・**repost_score**・**quote_score**・**share_score**・**dwel_time**

などはすべてポジティブな重みのシグナルとして扱われる。逆に

not_interested_score・**block_author_score**・**mute_author_score**・**report_score** はネガティブな重みを持つ。

つまり、1日にアクションをたくさんしても For You アルゴリズムで伸びにくくなる仕組みは見当たらない。むしろ履歴が蓄積されるほどモデルの精度が上がり、推薦に有利に働く設計になっている。

▶ よーん

ユーザーの直近32件のアクション履歴を Grok Transformer に入力する。
とあるが、ここで言っているユーザーとは、for you を見る人？

◀ Claude

コードで確認します。

◀ Claude

明確です。server.rs:66 で viewer_id として受け取り、それが query.user_id にセットされています。

For You を「見る人」（viewer）のアクション履歴です。

server.rs で：

viewer_id → query.user_id → user_action_sequence の取得に使用

つまり「自分が過去にいいね・リプライ・リポストした投稿の履歴」を Transformer に入力して、「この人は次にどんな投稿にエンゲージするか」を予測し、For You に流すコンテンツを選んでいきます。投稿者（author）の履歴ではありません。

▶ よーん

transformer

に入るアクションは、いいね、リプライ、リポスト、引用、ブックマーク、シェア、閲覧時間、not interested、ブロック、ミュート、通告 であって？ほかにある？

◀ Claude

全アクションが確認できました。

ユーザーが挙げたもの（いいね・リプライ・リポスト・引用・ブックマーク・シェア・閲覧時間・not interested・ブロック・ミュート・通報）はすべて含まれています。ただし「ブックマーク」は直接の名前では出てきません。

それ以外にコードに存在するアクション：

アクション 変数名

写真を展開（タップして拡大）`photo\_expand\_score`

投稿をクリック（詳細表示）`click\_score`

著者プロフィールをクリック `profile\_click\_score`

動画視聴 (VQV) `vqv\_score`

DM でシェア `share\_via\_dm\_score`

リンクコピーでシェア `share\_via\_copy\_link\_score`

引用された投稿をクリック `quoted\_click\_score`

著者をフォロー `follow\_author\_score`

動画視聴時間 (秒) `video\_watch\_time`

スクロール深度 `scroll\_depth`

合計で ACTIONS が18種 + CONTINUOUS_ACTIONS が **dwell_time** ・ **video_watch_time** ・ **scroll_depth** という構成です。

「ブックマーク」については **share_score** に含まれているか別途実装されている可能性があります、このファイル上では独立した変数名としては見当たりませんでした。

▶ よーん

エンゲージメントの定義は？

◀ Claude

重みの実際の値はリポジトリ内に定義されていません（実行時にパラメータストアから読み込む設計）。ただし、エンゲージメントの定義（スコアの計算式）は **weighted_scorer.rs** に明記されています。

コード上のエンゲージメントスコアの定義：

エンゲージメントスコア = Σ (各アクションの予測確率 × 重み)

具体的には以下のアクションの加重和：

アクション 重み定数 正負

いいね `FAVORITE\_WEIGHT` +

リプライ `REPLY\_WEIGHT` +

リポスト `RETWEET\_WEIGHT` +

写真展開 `PHOTO\_EXPAND\_WEIGHT` +

クリック (詳細表示) `CLICK\_WEIGHT` +

プロフィールクリック `PROFILE\_CLICK\_WEIGHT` +

動画視聴 (VQV) `VQV\_WEIGHT` +

シェア `SHARE\_WEIGHT` +

DM シェア `SHARE\_VIA\_DM\_WEIGHT` +

リンクコピー `SHARE\_VIA\_COPY\_LINK\_WEIGHT` +

滞在 (dwell) `DWELL\_WEIGHT` +

引用 `QUOTE\_WEIGHT` +

引用投稿クリック `QUOTED\_CLICK\_WEIGHT` +

滞在時間 (秒) `CONT\_DWELL\_TIME\_WEIGHT` +

著者フォロー `FOLLOW\_AUTHOR\_WEIGHT` +

興味なし `NOT\_INTERESTED\_WEIGHT` **--**

著者ブロック `BLOCK\_AUTHOR\_WEIGHT` **--**

著者ミュート `MUTE\_AUTHOR\_WEIGHT` **--**

通報 `REPORT\_WEIGHT` **--**

重みの具体的な数値はこのリポジトリには存在しません。ただし `offset_score()` の実装から、スコアが負になった場合は別の正規化ロジックが適用されることがわかります。

▶ よーん

ブックマークがシェアに含まれているかどうか調べて

◀ Claude

「bookmark」という語はリポジトリ内に一切存在しません。

`share_score` の内訳として `share_via_dm_score` (DM シェア) と `share_via_copy_link_score` (リンクコピー) は独立した変数として定義されていますが、ブックマークに対応する変数は見当たりません。

結論：このリポジトリ上では、ブックマークはシェアにも含まれておらず、アルゴリズムのシグナルとして定義されていない。

▶ よーん

ブックマークは for you アルゴリズムに効かないということ？

◀ Claude

断言はできません。理由は2つあります。

1. このリポジトリの範囲の問題

README には "core recommendation system" とあり、全システムが公開されているわけではありません。ブックマークの処理が別サービス（例：ユーザーアクションシーケンス取得の上流）で行われていて、このリポジトリに含まれていない可能性があります。

2. `share_score` の定義が不明

`share_score` ・ `share_via_dm_score` ・ `share_via_copy_link_score`

と3種類のシェアが独立して定義されているのに、ブックマークだけ変数がありません。ただし `share_score` が何を指すのかこのコードベース上では定義されておらず、ブックマークが含まれているかどうかは確認できません。

このリポジトリから言えること：

- ・ブックマークを独立したシグナルとして扱うコードは存在しない
- ・`bookmark` という語はリポジトリ内に一切ない

確定的に言えないこと：

- ・ブックマークが `share_score` に内包されているか
- ・上流のデータパイプラインでブックマークが別途処理されてアクション履歴に含まれているか

▶ よーん

X For You フィードアルゴリズム

このリポジトリは、X の「For You」フィードを動かすコア推薦システムを含んでいます。フォローしているアカウントからのコンテンツ（ネットワーク内）と、MLベースの検索で発見されたコンテンツ（ネットワーク外）を組み合わせ、Grokベースのトランスフォーマーモデルを使ってすべてをランキングします。

> 注記： トランスフォーマーの実装は、xAI による [Grok-1 オープンソースリリース](https://github.com/xai-org/grok-1) からポートし、推薦システム用途に改変したものです。

更新情報 — 2026年5月15日

今回のリリースでは For You アルゴリズムのコードを更新し、新しいコンポーネントとともに実行可能なエンドツーエンドの推論パイプラインを追加しました。

- エンドツーエンド推論パイプライン： 新しい **phoenix/run_pipeline.py** が、従来の **run_ranker.py** と **run_retrieval.py** を単一エントリーポイントに統合。エクスポート済みチェックポイントから 検索 → ランキング を実行し、本番環境での構成を再現します。
- 事前学習済みモデル成果物： 事前学習済みミニ Phoenix モデル（256次元埋め込み・4アテンションヘッド・2トランスフォーマー層）を Git LFS 経由で配布する約3GBのアーカイブとして同梱。自分でトレーニングせずに即座に推論が可能です。
- Grox コンテンツ理解パイプライン： 新しい **grox/** サービスを追加。スパム検出・投稿カテゴリ分類・PTOSポリシー適用などのコンテンツ理解タスク向けに、分類器・埋め込み器・タスク実行エンジンを提供します。
- 広告ブレンディングシステム： 新しい **home-mixer/ads/** モジュールを追加。フィード内への広告挿入・配置を処理し、センシティブなコンテンツの境界に配慮したブランドセーフティ追跡を含みます。
- クエリハイドレーター： ホームミキサーがフォロー中トピック・スターターパック・インプレッションブルームフィルター・IP・相互フォローグラフ・配信履歴などのユーザーコンテキストをハイドレートするようになりました。
- 候補ハイドレーター： エンゲージメント数・ブランドセーフティシグナル・言語コード・メディア検出・引用投稿展開・相互フォロースコアなどの追加ハイドレーターを追加。
- 候補ソース： 広告・フォロー候補・Phoenix MoE・Phoenix トピックス・プロンプト向けのソースを追加し、Thunder/Phoenix のソースを更新。

概要

For You フィードアルゴリズムは、2つのソースから投稿を取得・ランキング・フィルタリングします：

1. ネットワーク内（Thunder）：フォロー中のアカウントの投稿
2. ネットワーク外（Phoenix 検索）：MLで発見されたグローバルコーパスからの投稿

両ソースを統合し、Phoenix（Grokベースのトランスフォーマーモデル）で各投稿のエンゲージメント確率を予測してランキングします。最終スコアはこれら予測エンゲージメントの加重和です。

手動で設計したあらゆるフィーチャーと大半のヒューリスティクスをシステムから排除しました。Grokベースのトランスフォーマーが、あなたのエンゲージメント履歴（いいねしたもの、リプライしたもの、シェアしたものなど）を理解し、あなたに関連するコンテンツを決定することで、すべての重労働を担っています。

システムアーキテクチャ

（アーキテクチャ図は省略）

コンポーネント

ホームミキサー

場所： [home-mixer/](#)

For You フィードを組み立てるオーケストレーション層。 **CandidatePipeline** フレームワークを以下のステージで活用します：

ステージ 説明

クエリハイドレーター ユーザーコンテキスト（エンゲージメント履歴・フォローリスト）を取得

ソース Thunder と Phoenix から候補を取得

ハイドレーター 候補に追加データを付与

フィルター 不適格な候補を除去

スコアラー エンゲージメントを予測し最終スコアを算出

セレクター スコアでソートしトップKを選択

選択後フィルター 最終的な表示可否チェックと重複除去

サイドエフェクト 将来利用のためのリクエスト情報キャッシュ

サーバーは gRPC

エンドポイント（**ScoredPostsService**）を公開し、指定ユーザーのランキング済み投稿を返します。

Thunder

場所： [thunder/](#)

全ユーザーの最新投稿を追跡する、インメモリ投稿ストアとリアルタイム取り込みパイプライン。

- Kafka から投稿の作成・削除イベントを消費

- ・ユーザーごとにオリジナル投稿・リプライ/リポスト・動画投稿のストアを管理
- ・リクエストユーザーがフォローしているアカウントの「ネットワーク内」候補を提供
- ・保持期間を過ぎた投稿を自動トリミング

Thunder

により、外部データベースにアクセスせずサブミリ秒でネットワーク内コンテンツを検索できます。

Phoenix

場所: [phoenix/](#)

2つの主要機能を持つMLコンポーネント:

1. 検索 (ツータワーモデル)

関連するネットワーク外の投稿を発見:

- ・ユーザータワー: ユーザーフィーチャーとエンゲージメント履歴を埋め込みに変換
- ・候補タワー: 全投稿を埋め込みに変換
- ・類似度検索: ドット積類似度でトップKの投稿を取得

2. ランキング (候補分離型トランスフォーマー)

各候補のエンゲージメント確率を予測:

- ・ユーザーコンテキスト (エンゲージメント履歴) と候補投稿を入力
- ・候補同士が互いにアテンションできない特殊なマスクを使用
- ・各アクション種別 (いいね・リプライ・リポスト・クリックなど) の確率を出力

候補パイプライン

場所: [candidate-pipeline/](#)

推薦パイプライン構築のための再利用可能なフレームワーク。以下のトレイトを定義:

トレイト 目的

`Source` データソースから候補を取得

`Hydrator` 候補に追加フィーチャーを付与

`Filter` 表示すべきでない候補を除去

`Scorer` ランキング用スコアを算出

`Selector` トップ候補をソート・選択

`SideEffect` 非同期サイドエフェクト (キャッシュ・ログ) を実行

フレームワークは、設定可能なエラーハンドリングとログを備え、可能な場合はソースとハイドレーターを並列実行します。

仕組み

パイプラインステージ

1. クエリハイドレーション：

ユーザーの最近のエンゲージメント履歴とメタデータ（フォロワーリストなど）を取得

2. 候補ソーシング：以下から候補を取得：

- Thunder：フォロワー中のアカウントの最新投稿（ネットワーク内）
- Phoenix 検索：グローバルコーパスからMLで発見された投稿（ネットワーク外）

3. 候補ハイドレーション：候補を以下で補完：

- コア投稿データ（テキスト・メディアなど）
- 著者情報（ユーザー名・認証ステータス）
- 動画再生時間
- サブスクリプションステータス

4. スコアリング前フィルター：以下の投稿を除去：

- 重複
- 古すぎる投稿
- 自分自身の投稿
- ブロック・ミュートしたアカウント
- ミュートキーワードを含む投稿
- 以前見た投稿・直近のセッションで配信済みの投稿
- アクセス不可のサブスクリプションコンテンツ

5. スコアリング：複数のスコアラーを順番に適用：

- Phoenix スコアラー：Phoenix トランスフォーマーモデルからML予測を取得
- 重み付きスコアラー：予測を最終関連度スコアに統合
- 著者多様性スコアラー：フィードの多様性確保のため同一著者スコアを減衰
- OON スコアラー：ネットワーク外コンテンツのスコアを調整

6. 選択：スコアでソートしトップK候補を選択

7. 選択後処理：配信候補投稿の最終バリデーション

スコアリングとランキング

Phoenix Grokベーストランスフォーマーモデルは複数エンゲージメント種別の確率を予測します：

予測値：

- P(いいね)
- P(リプライ)
- P(リポスト)
- P(引用)
- P(クリック)
- P(プロフィールクリック)
- P(動画視聴)
- P(写真展開)
- P(シェア)

- P(滞在)
- P(著者フォロー)
- P(興味なし)
- P(著者ブロック)
- P(著者ミュート)
- P(通報)

重み付きスコアラーがこれらを最終スコアに統合します：

$$\text{最終スコア} = \sum (\text{重み}_i \times P(\text{アクション}_i))$$

ポジティブなアクション（いいね・リポスト・シェア）はプラスの重みを持ちます。ネガティブなアクション（ブロック・ミュート・通報）はマイナスの重みを持ち、ユーザーが嫌いそうなコンテンツを押し下げます。

フィルタリング

フィルターは2つのステージで実行されます：

スコアリング前フィルター：

フィルター 目的

- `DropDuplicatesFilter` 重複投稿IDを除去
- `CoreDataHydrationFilter` コアメタデータの取得に失敗した投稿を除去
- `AgeFilter` 閾値より古い投稿を除去
- `SelfpostFilter` ユーザー自身の投稿を除去
- `RepostDeduplicationFilter` 同一コンテンツのリポスト重複を除去
- `IneligibleSubscriptionFilter` アクセスできない有料コンテンツを除去
- `PreviouslySeenPostsFilter` すでに見た投稿を除去
- `PreviouslyServedPostsFilter` セッション内で配信済みの投稿を除去
- `MutedKeywordFilter` ミュートキーワードを含む投稿を除去
- `AuthorSocialgraphFilter` ブロック・ミュート済み著者の投稿を除去

選択後フィルター：

フィルター 目的

- `VFilter` 削除済み・スパム・暴力・グロコンテンツを除去
- `DedupConversationFilter` 同一会話スレッドの複数ブランチを重複除去

主要設計判断

1. 手動フィーチャーエンジニアリングなし

システムはGrokベースのトランスフォーマーがユーザーエンゲージメントシーケンスから関連度を学習することに完全に依存しています。コンテンツ関連度のための手動フィーチャーエンジニアリングは行いません。これによりデータパイプラインと配信インフラの複雑さが大幅に削減されます。

2. ランキング時の候補分離

トランスフォーマー推論中、候補同士は互いにアテンションできず、ユーザーコンテキストのみにアテンションします。これにより投稿のスコアがバッチ内の他の投稿に依存なくなり、スコアが一貫してキャッシュ可能になります。

3. ハッシュベースの埋め込み

検索とランキングの両方で、埋め込み参照に複数のハッシュ関数を使用します。

4. マルチアクション予測

単一の「関連度」スコアを予測する代わりに、モデルは多数のアクション種別の確率を予測します。

5. コンポーザブルなパイプラインアーキテクチャ

candidate-pipeline クレートは推薦パイプライン構築のための柔軟なフレームワークを提供します：

- ・パイプライン実行・監視とビジネスロジックの分離
- ・独立したステージの並列実行と丁寧なエラーハンドリング
- ・新しいソース・ハイドレーション・フィルター・スコアラーの容易な追加

ライセンス

このプロジェクトは Apache License 2.0 の下でライセンスされています。詳細は LICENSE を参照してください。

▶ よーん

*** X の for you アルゴリズムでは、投稿者が1日にポスト関連アクション（ポスト、リプライ、リポスト、引用リポスト）をすればするほど、ポストが伸びにくくなる？ Author diversity dampening に関することと思うんだけど**

◀ Claude

Author Diversity Dampening のコードを詳しく確認します。

◀ Claude

コードから仕組みが明確にわかりました。

結論：「1日のアクション数が多いほど伸びにくくなる」仕組みではない。

Author Diversity Dampening が実際にやっていることは全く別のことです。

何をしているか

対象：1回のフィード配信レスポンス内に同じ著者の投稿が複数入った場合

```
// 著者ごとに「この配信で何件目か」をカウント
let position = author_counts[author_id]; // 0始まり
```

```
// スコアに乗算する係数
multiplier = (1 - floor) × decay_factor^position + floor
```

同一著者の何件目か multiplier

1件目 (position=0) `decay\_factor^0 = 1.0` (減衰なし)

2件目 (position=1) `decay\_factor^1` (減衰)

3件目 (position=2) `decay\_factor^2` (さらに減衰)

ユーザーの質問への答え

想定していた仕組み 実際

1日にたくさん投稿するとペナルティ **そういう仕組みはない**

グローバルな日次アクション数が影響する **そういう仕組みはない**

実際の動作は「ある1人のユーザーのフィードに、あなたの投稿が複数候補として上がってきたとき、2件目以降のスコアが下がる」というものです。

つまり影響を受けるのは「同じ視聴者のフィードに同時に複数登場するかどうか」であり、1日に何回投稿したかとは直接関係ありません。1日10回投稿しても、それぞれが別のユーザーのフィードに1件ずつ入るなら減衰は起きません。

▶ よーん

1回のフィードというのは、for you 表示リクエスト1回においてということ？

◀ Claude

そのとおりです。

コードを見ると：

```
async fn score(
    &self,
    _query: &ScoredPostsQuery, // ← 1回のリクエスト
    candidates: &[PostCandidate], // ← そのリクエストで返す候補群
) -> Result<Vec<PostCandidate>, String> {
    let mut author_counts: HashMap<u64, usize> = HashMap::new(); // ←
    リクエストごとに初期化
```

author_countsはこの関数が呼ばれるたびに新規作成されます。つまりリクエストをまたいで状態は持ち越されません。

実際の動作イメージ：

- ユーザーが For You を開く → 1リクエスト → 候補をまとめてスコアリング → 同一著者の2件目以降を減衰 → 返す
- スクロールして追加読み込み → 別リクエスト → **author_counts** はまたゼロから

なので「1日に何回投稿するか」ではなく、「1回のフィード読み込みで同じ著者の投稿が複数候補に上がるか」が減衰の発動条件です。

▶ よーん

例えば、1つメイン投稿して、それに対するリプライにリプライを30件ぐらいして、他のひとのポストを30件ぐらいリポストしたとする。すると、本来伸ばしたいはずのメイン投稿が、リプライやリポストのせいでfor you ni

▶ よーん

[Request interrupted by user]

▶ よーん

例えば、1つメイン投稿して、それに対するリプライにリプライを30件ぐらいして、他のひとのポストを30件ぐらいリポストしたとする。すると、本来伸ばしたいはずのメイン投稿が、リプライやリポストのせいでfor you にひょうじされにくくなるというのはいりうる？

◀ Claude

コードで確認します。リプライ・リポストが候補プールに入るかどうかを調べます。

◀ Claude

重要なことがわかりました。

Thunder が投稿を3種類に分けて管理しています：

```
let is_original = !post.is_reply && !post.is_retweet;

// オリジナル投稿（リプライでもリポストでもないもの）
original_posts_by_user

// リプライ・リポスト（セカンダリ）
secondary_posts_by_user

// 動画投稿
video_posts_by_user
```

そして候補プールには両方が入ります（ただし別々に上限あり：**MAX_ORIGINAL_POSTS_PER_AUTHOR**、**MAX_REPLY_POSTS_PER_AUTHOR**）。

ご質問への答え：直接的には伸びにくくならない。

理由はスコアリングの順序にあります：

Author Diversity Scorer はスコアの高い順に処理します。

```
// まず weighted_score で降順ソート
ordered.sort_by(|(a), (b)| {
    b_score.partial_cmp(&a_score) // 高スコア順
});

// その順番で著者ごとの position をカウント
let position = author_counts[author_id]; // 0始まり
```

つまり：

投稿 スコア順位 position 減衰

メイン投稿（オリジナル） 1位 0 **なし（×1.0） **

リプライ群 それ以下 1, 2, 3... **減衰される**

リポスト群 それ以下 続き **さらに減衰**

メイン投稿がその著者の中で最高スコアであれば、**position=0**になるためフルスコアが維持されます。減衰を受けるのはリプライとリポストの方です。

ただし例外があります：

もし何らかのリポストがメイン投稿より高いスコアを持った場合、そのリポストが**position=0**（減衰なし）になり、メイン投稿が**position=1**（減衰あり）に押し下げられます。ただし通常、オリジナル投稿の方がリプライより高スコアになりやすい設計です。

▶ よーん

動画投稿とオリジナル投稿はなんでthunder で別々に扱われるの？

◀ Claude

取得の仕方を確認します。

◀ Claude

理由がコードから明確にわかりました。

動画専用のフィードを別途配信するためです。

thunder_service.rs の核心部分：

```
let all_posts: Vec<LightPost> = if req.is_video_request {  
    // 動画ストアから取得  
    post_store.get_videos_by_users(...)  
} else {  
    // オリジナル + セカンダリ（リプライ・リポスト）から取得  
    post_store.get_all_posts_by_users(...)  
};
```

is_video_request

フラグで完全に別ルートを通ります。返せる件数の上限も別定数（**MAX_VIDEOS_TO_RETURN** vs **MAX_POSTS_TO_RETURN**）で管理されています。

ただし現在の home-mixer の ThunderSource では：

```
is_video_request: false, // 常に false
```

つまり通常の For You フィードでは動画ストアは使われておらず、動画専用フィード（Xの「動画」タブのようなもの）向けに別途リクエストされる設計になっています。

まとめると：

- **original_posts_by_user** + **secondary_posts_by_user** → 通常の For You フィード
- **video_posts_by_user** → 動画専用フィード

同じデータを用途別に分けてストアすることで、動画フィードのリクエスト時に全投稿をスキャンしなくて済む（効率化）という設計です。

▶ よーん

Author Diversity Scorer はスコアの高い順に処理します。 とあるけど、このスコアとは何？

◀ Claude

コードを見ると：

```
ordered.sort_by(|(a), (b)| {  
    let a_score = a.weighted_score.unwrap_or(f64::NEG_INFINITY);  
    let b_score = b.weighted_score.unwrap_or(f64::NEG_INFINITY);  
    b_score.partial_cmp(&a_score)  
});
```

weighted_score を使っています。これは直前のステップ（Weighted Scorer）が計算した値です。

Final Score = Σ (各アクションの予測確率 × 重み)

つまり：

```
P(いいね) × FAVORITE_WEIGHT  
+ P(リブライ) × REPLY_WEIGHT  
+ P(リポスト) × RETWEET_WEIGHT  
+ P(クリック) × CLICK_WEIGHT  
+ ...  
- P(ブロック) × BLOCK_AUTHOR_WEIGHT  
- P(ミュート) × MUTE_AUTHOR_WEIGHT  
- ...
```

「この投稿を見たユーザーがどれだけエンゲージしそうか」をトランスフォーマーが予測した確率の加重和です。Author Diversity Scorer はこのスコアの降順に処理し、同一著者の2件目以降を減衰させます。

▶ よーん

**スコアリング前フィルター：以下の投稿を除去：重複 古すぎる投稿
で、重複と古すぎる投稿の定義は？**

◀ Claude

コードで確認します。

◀ Claude

MAX_POST_AGE の具体的な数値はこのリポジトリに定義されていません（外部の設定から読む形）。ただし2点はコードから明確です。

重複の定義

```
let mut seen_ids = HashSet::new();  
for candidate in candidates {  
    if seen_ids.insert(candidate.tweet_id) { // 初出なら通す  
        kept.push(candidate);  
    } else {  
        removed.push(candidate); // 2件目以降は除去  
    }  
}
```

同じ **tweet_id** が複数のソース（Thunder と Phoenix 両方）から候補として上がってきた場合、2件目以降を除去します。内容の重複ではなく、ID の重複です。

古すぎる投稿の定義

```
duration_since_creation_opt(tweet_id)
.map(|age| age <= self.max_age) // max_age 以内なら通す
```

投稿年齢は **tweet_id**（Snowflake ID）からタイムスタンプを抽出して計算します。閾値は **MAX_POST_AGE**（秒）ですが、この値はリポジトリ内に定義がなく、外部パラメータです。

参考として Thunder のデフォルト保持期間はコード上 2日間（**2 * 24 * 60 * 60** 秒）と書かれているので、For You の上限もその前後と推測されますが、確定はできません。

▶ よーん

tweet_id が複数のソース（Thunder と Phoenix 両方）から候補として上がってきた場合、2件目以降を除去します。とあるけど、thnder と phonix から来る候補のポストって、排他的なんじゃないの？ネットワーク内のアカウントとネットワーク外のアカウントにたいおうしてるんでしょ？

◀ Claude

鋭い指摘です。コードで確認します。

◀ Claude

ソースが実は6つあることがわかりました。Thunder と Phoenix だけではありません。

```
let sources = vec![
  thunder_source, // フォロー中アカウントの投稿
  tweet_mixer_source, // 別の推薦システム
  phoenix_source, // Phoenix MLによるネットワーク外
  phoenix_topics_source, // トピックベースのPhoenix
  phoenix_moe_source, // Phoenix Mixture of Experts
  cached_posts_source, // キャッシュ済み投稿
];
```

重複が起きうる理由

ユーザーの指摘は正しくて、Thunder（ネットワーク内）と Phoenix（ネットワーク外）は基本的に別です。ただし重複が起きうるケースがあります：

- **phoenix_source** ・ **phoenix_topics_source** ・ **phoenix_moe_source** は3つとも独立した ML 検索なので、同じ投稿を別々に引いてくることがある
- **tweet_mixer_source** はさらに別の推薦システムで、Phoenix と同じ投稿を返すことがある
- フォローしているアカウントの投稿が非常にバズっていれば、Thunder と Phoenix の両方から候補として上がる可能性もゼロではない

そのため **DropDuplicatesFilter** は「念のため」ではなく、複数の独立したソースが同じ **tweet_id** を返す構造的な可能性に対処するために必要な設計です。

▶ よーん

Phoenix スコアラー：Phoenix トランスフォーマーモデルからML予測を取得 とあるけど、thunder から上がってきたポストも、phoenix トランスフォーマーでスコアリングしているの？

▶ よーん

This session is being continued from a previous conversation that ran out of context. The summary below covers the earlier portion of the conversation. Summary: 1. Primary Request and Intent: - Session started by continuing previous work on `record-note-posts` skill: updating column structure (A=投稿日時, B=記事URL, C=タイトル) and fixing wrong account ID (takaesu7731→takaesu7431) - User requested: (1) change column A back to 投稿日時, (2) implement 文字数 fetching from note article body - `/record` skill invoked: created report for record-note-posts新設, added 2 changelog entries - User cloned `https://github.com/xai-org/x-algorithm.git` to `/root/x-algorithm` - User wants to ask questions about X's For You algorithm using ONLY this repo (no web search) - Series of algorithm questions about the For You feed 2. Key Technical Concepts: - **note.com unofficial APIs**: v2/creators/{urlname}/contents (articles), v1/stats/pv?filter=all (cumulative stats), v3/notes/{key} (article body HTML) - **fetch_note_stats.py**: fetches note articles with charCount via HTML body stripping - **X For You Algorithm** (x-algorithm repo): - Phoenix: Grok-based Transformer for scoring (retrieval + ranking) - Thunder: in-memory store for in-network posts (from followed accounts) - Home Mixer: orchestration layer - 6 candidate sources: thunder, tweet_mixer, phoenix, phoenix_topics, phoenix_moe, cached_posts - Viewer's 32-action history as Transformer input (not author's) - 18 action types + continuous actions (dwell_time, video_watch_time, scroll_depth) - Engagement score = weighted sum of predicted action probabilities - Author Diversity Dampening: per-request, score-ordered, dampens 2nd+ posts from same author - Thunder stores: original_posts_by_user, secondary_posts_by_user, video_posts_by_user - DropDuplicatesFilter: deduplicates by tweet_id across all sources - AgeFilter: removes posts older than MAX_POST_AGE (value not in repo) 3. Files and Code Sections: - `/root/xClaude/scripts/fetch\_note\_stats.py` - Added `import html` - Added `fetch\_body\_length(key, headers)` function using `/api/v3/notes/{key}` - HTML tag removal: `re.sub(r"]>", "", body)`, then `html.unescape()`, then whitespace removal - Added `"charCount": char\_count` to results - `/root/xClaude/.claude/skills/record-note-posts/SKILL.md` - Column order updated: A=投稿日時, B=記事URL, C=タイトル, D=文字数(from v3 API), E=ハッシュタグ, F=サムネURL, G=サムネプレビュー, H=ビュー, I=スキ, J=スキ率 - STEP 2: range changed to B:B (記事URL) - STEP 3: updated to H・I・J列 - STEP 5 append values order updated to match new column structure - `/root/xClaude/docs/changelog.md` - added 2 entries for record-note-posts新設 and シート名変更 - `/root/xClaude/docs/reports/20260516\_record\_note\_posts\_skill.md` - new report created - `/root/x-algorithm/phoenix/runners.py` - ACTIONS list (18 types), CONTINUOUS_ACTIONS - `/root/x-algorithm/home-mixer/scorers/weighted\_scorer.rs` - engagement score formula - `/root/x-algorithm/home-mixer/scorers/author\_diversity\_scorer.rs` - diversity dampening logic - `/root/x-algorithm/home-mixer/posts/post\_store.rs` - Thunder storage (original/secondary/video separation) - `/root/x-algorithm/home-mixer/thunder\_service.rs` - is_video_request routing - `/root/x-algorithm/home-mixer/filters/drop\_duplicates\_filter.rs` -

dedup by tweet_id - `/root/x-algorithm/home-mixer/filters/age\_filter.rs` - age filter using MAX_POST_AGE -

`/root/x-algorithm/home-mixer/candidate\_pipeline/phoenix\_candidate\_pipeline.rs` - 6 sources combined

4. Errors and fixes: - `/api/v2/notes/{key}` returned 404 → fixed by using `/api/v3/notes/{key}` which returns 200 - `/api/v1/notes/{key}` returned 405 (method not allowed) - `mcp\_\_github\_\_push\_files` tool not available → used `git push origin master` directly (worked)

5. Problem Solving: - 文字数取得: confirmed v3 API endpoint works, implemented HTML stripping for accurate character count - Sheet re-population: cleared rows 2-25, re-fetched all 24 articles with new structure, bulk updated - /record skill: found 2 unrecorded changes, created report + changelog entries, committed and pushed

6. All user messages: - "やっぱり投稿日時をA列に変更。文字数は取得または本文からカウントできない?" - "はい" (to implement 文字数) - "/record" - "1 は変更ログと報告書。2は変更ログ" - "https://github.com/xai-org/x-algorithm.git をclone して" - "そのリポジトリの情報だけを参考にして僕の質問に答えて。web search は禁止。" - "X の for you アルゴリズムでは、1日にアクション（ポスト、リプライ、リポスト、引用リポスト、ブックマーク、いいね）をすればするほど、ポストが伸びにくくなる?" - "ユーザーの直近32件のアクション履歴を Grok Transformer に入力する。 とあるが、ここで言っているユーザーとは、for you を見る人?" - "transformer

に入るアクションは、いいね、リプライ、リポスト、引用、ブックマーク、シェア、閲覧時間、not interested, ブロック、ミュート、通告 であって?ほかにもある?" - "エンゲージメントの定義は?" - "ブックマークがシェアに含まれているかどうか調べて" - "ブックマークは for you アルゴリズムに効かないということ?" - "README を和訳して" - "例えば、1つメイン投稿して、それに対するリプライにリプライを30件ぐらいして、他のひとのポストを30件ぐらいリポストしたとする。すると、本来伸ばしたいはずのメイン投稿が、リプライやリポストのせいでfor you にひょうじされにくくなるというのはありうる?" - "1回のフィードというのは、for you 表示リクエスト1回においてということ?" - "動画投稿とオリジナル投稿はなんでthnder で別々に扱われるの?" - "Author Diversity Scorer はスコアの高い順に処理します。 とあるけど、このスコアとは何?" - "スコアリング前フィルター:

以下の投稿を除去: 重複と古すぎる投稿の定義は?" - "tweet_id が複数のソース (Thunder と Phoenix 両方) から候補として上がってきた場合、2件目以降を除去します。 とあるけど、thnder と phoenix から来る候補のポストって、排他的なんじゃないの?ネットワーク内のアカウントとネットワーク外のアカウントにたいおうしてるんでしょ?" - "Phoenix スコアラー: Phoenix トランスフォーマーモデルからML予測を取得 とあるけど、thunder から上がってきたポストも、phoenix トランスフォーマーでスコアリングしているの?"

7. Pending Tasks: - Answer the current question: "thunder から上がってきたポストも、phoenix トランスフォーマーでスコアリングしているの?"

8. Current Work: The user just asked: "Phoenix スコアラー: Phoenix トランスフォーマーモデルからML予測を取得 とあるけど、thunder から上がってきたポストも、phoenix トランスフォーマーでスコアリングしているの?" This question asks whether Thunder's in-network posts also go through Phoenix transformer scoring. From the pipeline structure in `phoenix\_candidate\_pipeline.rs`, all sources (including thunder_source) feed candidates into the same pipeline, and the scoring stage (PhoenixScorer → WeightedScorer → AuthorDiversityScorer) is applied to ALL candidates regardless of source. The sources are just listed in a vec and their outputs are all merged before filtering and scoring.

9. Optional Next Step: Answer the user's question: "thunder から上がってきたポストも、phoenix

トランスフォーマーでスコアリングしているの?" The answer is YES - Thunder posts go through the same Phoenix transformer scoring as Phoenix-retrieved posts. This is because in `phoenix\_candidate\_pipeline.rs`, all 6 sources feed into one combined candidate pool, and the PhoenixScorer is applied to all candidates in that pool regardless of origin. There is no separate scoring path for in-network vs out-of-network candidates at the transformer stage. If you need specific details from before compaction (like exact code snippets, error messages, or content you generated), read the full transcript at: [/root/.claude/projects/-root-xClaude/802b7cfb-6ec6-46c3-873d-4283aa51a2d8.jsonl](#) Continue the conversation from where it left off without asking the user any further questions. Resume directly — do not acknowledge the summary, do not recap what was happening, do not preface with "I'll continue" or similar. Pick up the last task as if the break never happened.

◀ Claude

はい、Thunder から上がってきた投稿も Phoenix Transformer でスコアリングされます。

`phoenix_candidate_pipeline.rs` の構造では、6つのソース（Thunder含む）の出力がすべて1つの候補プールにマージされてから、一括でスコアリングステージに渡されます。スコアリングは候補の出所を区別しません。

つまりパイプラインは：

```
[Thunder] ↘  
[TweetMixer] → 全候補をマージ → PhoenixScorer (全員に適用) → WeightedScorer →  
AuthorDiversityScorer  
[Phoenix] ↗  
...
```

という流れです。

ただし、Thunder投稿がPhoenixTransformerで再スコアされる意味は何かというと：

- Thunder はリアルタイムの時系列ソート (`score_recent`) で候補を出す
- Phoenix Transformer はその候補に対して「このユーザーがエンゲージする確率」を予測する

つまり Thunder

は「フォロー中の誰が最近何を投稿したか」を素早く引いてくる役割で、最終的なランキングは Phoenix Transformer の ML 予測スコアで決まります。Thunder のソート順はあくまで候補絞り込みのためで、For You の最終表示順を決めるのは Phoenix です。

▶ よーん

Grok と phoenix の関係は？ thunderは単なるデータベース？

◀ Claude

リポジトリを確認して正確に答えます。

◀ Claude

整理します。

Grok と Phoenix の関係

Grok = Transformer アーキテクチャそのもの (**grok.py**)

- Multi-Head Attention + RoPE + Decoder Layer を実装した純粋なニューラルネット
- 汎用の Transformer であり、言語モデルとしても推薦モデルとしても使える

Phoenix = Grok を backbone に使った推薦モデル (**recsys_model.py**)

- **grok.py** から **Transformer**, **TransformerConfig** 等を import している
- 推薦用の前処理を追加：
 - ユーザー埋め込み (1トークン)
 - 過去の行動履歴 (最大 128 トークン)
 - 候補ポスト (最大 32 トークン)
- これら3つを結合して Grok の Transformer に入力
 - 候補同士はお互いに attention しない特殊マスクを使う (各候補を独立にスコアリング)
 - 出力: 候補ごとにアクション確率のベクトル

つまり「Phoenix とは Grok 製 Transformer を使った推薦エンジン」です。

Thunder は単なるデータベース？

半分正解、半分違います。

Thunder の実態: Kafka-fed リアルタイム in-memory キャッシュ

- Kafka からツイートイベントを購読して、フォロー中ユーザーの最新ポストをメモリ上に保持
- ユーザーID でポストを O(1) で引き出せる構造
- **score_recent** (時刻順ソート) はするが、ML スコアリングはしない
- gRPC サービスとして動作し、Home Mixer からリクエストを受け付ける

データベースとの違い:

- 永続化しない (メモリのみ)
- Kafka を通じてリアルタイムで更新され続ける
- 単純な時刻ソートで「フォロー中の最新ポスト候補」を高速に提供する役割

全体の役割分担まとめ

Thunder → 「フォロー中の誰が最近何を投稿したか」を高速に引く (リアルタイム in-memory)

Phoenix → 候補ポストに「このユーザーがエンゲージする確率」を ML で予測する

Grok → Phoenix の中で動く Transformer 本体

Grox → ポスト埋め込み・スパム判定・安全性チェックなどを担う別システムのエンジン

補足: 以前「直近32件のアクション履歴」と説明しましたが、**PhoenixModelConfig** を見ると **history_seq_len = 128** (最大128) が正しい値です。

▶ よーん

Kafka を通じてリアルタイムで更新され続ける とは？

◀ Claude

Thunder の Kafka 周りのコードを確認します。

◀ Claude

コードから仕組みが完全にわかりました。図解します。

Kafka とは

X 上のあらゆるイベント（投稿・削除など）をリアルタイムに流すメッセージバスです。「出来事が起きたら即座にキューに積む」仕組み。

Thunder の更新フロー

```
誰かが X に投稿
↓
Kafka トピックに
TweetCreateEvent が積まれる
↓
Thunder の Kafka Consumer が
常時 poll() でメッセージを取得
↓
LightPost に変換
↓
post_store.insert_posts() で
メモリ上のマップに書き込む

誰かが削除した場合 → post_store.mark_as_deleted()
```

コードでは `process_tweet_events_v2` (`tweet_events_listener_v2.rs:181`) が無限ループで Kafka をポーリングし続けています。

```
loop {
  let messages = consumer.poll(batch_size).await;
  // → LightPost に変換して post_store に insert
}
```

まとめ

Thunder はサーバー起動時からバックグラウンドスレッドが Kafka を監視し続けており、誰かが投稿するたびにメモリ内のポストマップが更新されます。

Home Mixer から「この人のフォロー中の最新ポストをくれ」というリクエストが来たとき、Thunder はそのメモリマップを即座に返します。DBアクセスなし、ゼロ遅延に近い形で。

▶ よーん

このメモリとはどこのメモリ？Xのサーバー？スマホ？

◀ Claude

X のサーバー（データセンター）上のRAMです。

main.rs を見ると、Thunder は gRPC サーバーとして起動するプログラムです：

```
let post_store = Arc::new(PostStore::new(...)); // ← サーバードメインのRAMに確保

// gRPC サーバーとして起動
let http_server = HttpServer::new(args.http_port, ...).await;
```

つまり構造はこうです：

```
[スマホ/PC]
↓ HTTPS リクエスト
[Xのサーバー群（データセンター）]
├── Home Mixer サーバー
├── Thunder サーバー ← このRAMにポストが乗っている
├── Phoenix サーバー
└── Kafka クラスター
```

スマホはあくまで「For You を見せて」とリクエストを送るだけ。候補取得・スコアリング・フィルタリングはすべてサーバー側で完結しています。

スマホのアプリ内にアルゴリズムはありません。

▶ よーん

全アカウントのポスト記録が thunder のメモリに載っているの？

◀ Claude

config.rs

はリポジトリに含まれていないため定数の実際の値は不明ですが、コードから構造はわかります。

結論：全アカウントではなく「最近投稿したアカウントの直近N日分」

Thunder が保持するのは：

時間制限あり（**retention_seconds**、デフォルト 2日間）

```
// Default::default() より
Self::new(2 * 24 * 60 * 60, 0) // 2日 = 172800秒
```

古いポストは2分ごとに自動削除（**start_auto_trim(2)**）

著者ごとの上限あり

```
MAX_ORIGINAL_POSTS_PER_AUTHOR // オリジナル投稿の上限
MAX_REPLY_POSTS_PER_AUTHOR // リプライの上限
MAX_VIDEO_POSTS_PER_AUTHOR // 動画の上限
```

（実際の数値は非公開）

全体像

Thunder メモリの中身

= 「直近2日以内に投稿したアカウント」 × 「著者ごとN件まで」

ただし X の規模（月間アクティブ数億人）だと、それだけでも膨大な量になります。現実には Thunder は複数台に分散して動いているはずです（このコードは1インスタンス分の実装）。

スマホが For You を開いたとき、Home Mixer は「このユーザーのフォロー先リスト」を Thunder に渡し、Thunder はそのリストにマッチするユーザーのポストだけを返します。Thunder 全体に載っている全ポストが返ってくるわけではありません。

▶ よーん

Kafka クラスターってのは何？

◀ Claude

Kafka とは「巨大なリアルタイム伝言板」

X 上では毎秒数十万件のイベント（投稿・いいね・削除など）が発生します。それを各サービスに届けるためのメッセージバスが Kafka です。

仕組み

[投稿サーバー]

↓ 「新しいポストが来たよ」というメッセージを書き込む

| Kafka クラスター | ← X のデータセンター内のサーバー群
| （メッセージを一時保存） |

↓ 読み出す (poll)

[Thunder] ← 「最新ポストをメモリに入りたい」

[検索サービス] ← 「全文検索インデックスを更新したい」

[通知サービス] ← 「フォロワーに通知したい」

誰かが1回投稿すれば、興味のある全サービスが同じメッセージを受け取れる。投稿サーバーが Thunder・検索・通知…を個別に叩く必要がない。

「クラスター」とは

Kafka 単体だと1台のサーバーが壊れたら止まります。クラスター = Kafka を複数台で冗長化したもの。メッセージを複数台に複製して持つことで、1台落ちても止まらない。

Thunder との関係（コードで確認した事実）

Thunder には2つのモードがありました（`kafka_utils.rs`）：

非serving モード (Feeder) :

生のツイートイベント Kafka トピック → 読む → 整形 → in-network 用トピックに書く

serving モード:

in-network 用トピック → 読む → メモリに格納 → gRPC で Home Mixer に返す

つまり Thunder の前段にも「生イベントを加工して流す」別プロセスがあります。Kafka がその2段階をつなぐ接着剤になっています。

▶ よーん

phoenix のネットワーク外のポスト検索では、thunder上の全ポストを対象に検索を行うのですか？

◀ Claude

Phoenix の検索ロジックを確認します。

◀ Claude

Thunder を使いません。Phoenix の検索は全く別のインデックスを持っています。

コードの核心はここです（[recsys_retrieval_model.py:381](#)）：

```
# corpus_embeddings = [N, D] の行列（事前に計算済みのポスト埋め込み群）
scores = jnp.matmul(user_representation, corpus_embeddings.T)
top_k_scores, top_k_indices = jax.lax.top_k(scores, top_k)
```

Phoenix 検索の仕組み（2タワー検索）

```
[ユーザー]
行動履歴128件 → User Tower (Transformer) → ユーザーベクトル [D次元]
↓ 内積
[コーパス] ← 事前計算済みの全候補ポスト埋め込み [N×D]
↓
スコア上位K件を返す
```

コーパス（[corpus_embeddings](#)）は Thunder とは別物です：

- Thunder = リアルタイム更新の「フォロー中ユーザーの最新ポスト」
- Phoenix コーパス = 事前にバッチ処理で作った「全候補ポストの埋め込みインデックス」

[run_retrieval.py](#) でも [set_corpus\(\)](#) で別途セットしています：

```
inference_runner.set_corpus(corpus_embeddings, corpus_post_ids)
```

まとめ

データソース 更新方式 対象

Thunder RAM上のポストマップ Kafka リアルタイム フォロー中のみ

Phoenix コーパス 埋め込みインデックス バッチ（定期更新） ネットワーク外も含む

Phoenix はユーザーを1本のベクトルに圧縮し、そのベクトルとコーパス全体の内積を一気に計算することで、全候補を「ブルートフォースで1件ずつ評価」せずに高速検索しています。これを ANN（近似最近傍探索）と呼びます。

▶ よーん

全候補を「ブルートフォースで1件ずつ評価」せずに高速検索しています。これを ANN（近似最近傍探索）と呼びます。これがよくわからないです

◀ Claude

まず「普通の検索」から考える

ポストが1億件あるとして、あなたに合うポストを探す方法：

ブルートフォース（総当たり）

```
ポスト1 と あなた → スコア計算
ポスト2 と あなた → スコア計算
ポスト3 と あなた → スコア計算
...
ポスト1億件目 → スコア計算
→ 一番高いやつを返す
```

正確だが遅い。

Phoenix がやっていること

まず、すべてのポストを「意味の地図」の上の点に変換しておく。

```
[宇宙系のポスト群]
★★★
★★★
★
[政治系] ●●● [スポーツ系]
●● ◆◆◆
◆◆
```

ユーザーも同じ地図の上の点に変換する：

```
あなた → （宇宙・物理が好きな人）
```

「あなたの近くにある点＝あなたに合うポスト」として取り出す。

1億件全部と距離を測る必要がなく、地図の「このあたり」だけ見ればよい。

コードとの対応

```
# user_representation = あなたの座標ベクトル [D次元]
# corpus_embeddings = 全ポストの座標一覧 [N×D]

scores = jnp.matmul(user_representation, corpus_embeddings.T)
```

この行列積が「あなたの座標と全ポストの座標の近さ（内積）を一括計算」です。

実際には「完全に近い点」ではなく「だいたい近い点」を高速に見つけるため 近似（Approximate）と呼ばれます。多少精度を下げる代わりに、1億件でも数ミリ秒で返せます。

「2タワー」と呼ばれる理由

```
ユーザー側の塔（User Tower） ポスト側の塔（Candidate Tower）
行動履歴 → Transformer 投稿内容+著者 → MLP
↓ ↓
ユーザーベクトル ポストベクトル（事前計算）
↘ ↙
同じ「意味の地図」上で比較
```

ポスト側は事前に計算して保存しておけるので、リクエストのたびにやり直す必要がない。ユーザー側だけリアルタイムで計算すればよい。これが高速化の核心です。

▶ よーん

phoenix 検索では、ネットワーク内のポストも返される可能性もあるわけ？

◀ Claude

コードを確認します。

◀ Claude

はい、あります。コードがそれを証明しています。

3つの証拠

① PhoenixSource に「ネットワーク外のみ」という制限がない（[phoenix_source.rs:63](#)）

```
fn enable(&self, query: &ScoredPostsQuery) -> bool {
    !query.in_network_only // ← Following タブでは動かないが
    && !query.has_cached_posts
    // ← 「フォロー外のみ」という条件は存在しない
}
```

② Phoenix 候補に「in_network」フラグを後付けで判定している（[in_network_candidate_hydrator.rs:32](#)）

```
let is_in_network = is_self || followed_ids.contains(&candidate.author_id);
// Phoenix が返した候補も含め、全候補に対してフォロー中かどうかを判定
```

もしフォロー中のポストが最初からコーパスに入っていないなら、このチェック自体が不要なはず。

③ OON スコアラーがフォロー外候補のスコアを下げる（[oon_scorer.rs:21](#)）

```
Some(false) => base_score * p::OON_WEIGHT_FACTOR, // OON は減点
_ => base_score, // in_network は減点なし
```

まとめ

```
Phoenix コーパス
= フォロー中のポスト も フォロー外のポスト も 両方含む

↓ Phoenix が返した後

InNetworkCandidateHydrator が「フォロー中？」を判定
```

OON Scorer がフォロー外のスコアを OON_WEIGHT_FACTOR 倍に下げる
DropDuplicatesFilter が Thunder と Phoenix の重複を除去

つまり Phoenix がフォロー中のポストを返しても弾かれるわけではなく、むしろ OON の減点を受けない分だけ有利に扱われます。Thunder と同じポストが来た場合は重複除去で1件になります。

▶ よーん

重み付けスコアを出す時に使う、各アクションの重みを教えて

◀ Claude

params.rs はリポジトリに含まれていません。実際の重みの数値は非公開です。

わかっていること

重みが存在するアクションの一覧は `weighted_scorer.rs` から確認できます：

アクション 定数名 正/負

いいね `FAVORITE\_WEIGHT` 正

リプライ `REPLY\_WEIGHT` 正

リポスト `RETWEET\_WEIGHT` 正

画像展開 `PHOTO\_EXPAND\_WEIGHT` 正

クリック `CLICK\_WEIGHT` 正

プロフィールクリック `PROFILE\_CLICK\_WEIGHT` 正

動画視聴（長尺のみ） `VQV\_WEIGHT` 正

シェア `SHARE\_WEIGHT` 正

DM でシェア `SHARE\_VIA\_DM\_WEIGHT` 正

リンクコピー `SHARE\_VIA\_COPY\_LINK\_WEIGHT` 正

滞在時間（discrete） `DWELL\_WEIGHT` 正

引用RT `QUOTE\_WEIGHT` 正

引用RTのクリック `QUOTED\_CLICK\_WEIGHT` 正

滞在時間（continuous） `CONT\_DWELL\_TIME\_WEIGHT` 正

フォロー `FOLLOW\_AUTHOR\_WEIGHT` 正

興味なし `NOT\_INTERESTED\_WEIGHT` **負**

ブロック `BLOCK\_AUTHOR\_WEIGHT` **負**

ミュート `MUTE\_AUTHOR\_WEIGHT` **負**

報告 `REPORT\_WEIGHT` **負**

正/負の判断は `offset_score` 関数の構造（負スコアに対して別処理をしている）と各アクションの性質から明らかです。ただし具体的な数値は非公開で、このリポジトリには含まれていません。

▶ よーん

Replies (返信) > 引用リポスト > リポスト > いいね・滞在時間 という関係はこのリポジトリからはわからないのね

◀ Claude

そうです、具体的な数値はこのリポジトリには存在しません。

わかるのは：

- どのアクションに重みがある（定数名の一覧）
- 負のフィードバック（ブロック・ミュート・報告・興味なし）がスコアを下げる
- 動画は長尺（`MIN_VIDEO_DURATION_MS` 超）のときだけ `VQV_WEIGHT` が適用される
- ネットワーク外候補は `ON_WEIGHT_FACTOR` 倍される

「Replies > 引用 > リポスト > いいね」という順序は X
社が外部で公表した情報であり、このコードからは読み取れません。

▶ よーん

ポストに外部リンクを含んでいたら、スコアを減じられるということはある？

◀ Claude

このリポジトリには外部リンク含有によるスコア減点のロジックは存在しません。

確認した範囲：

- `weighted_scorer.rs` — リンクに関する重みや条件なし
- スコアラー群（`scorers/`） — リンク・URL を参照するコードなし
- フィルター群（`filters/`） — リンクによる除外ロジックなし
- 候補特徴量（`candidate_features.rs`） — URL/リンク系のフィールドなし

ただしこれは「このリポジトリに書かれていない」というだけです。

実際には `params.rs`（非公開）の中に URL
有無に応じた重み調整が隠れている可能性は排除できませんし、Phoenix のコーパス構築段階や Grox
の埋め込み生成段階でリンク付きポストが別扱いされている可能性もあります。

このリポジトリの情報だけでは、外部リンクによるペナルティの有無は断言できない、というのが正確な答えです。

▶ よーん

phoenix のコーパスにポストが保持される期間は？

◀ Claude

唯一見つかった具体的な数値は `recsys_model.py:33` にあります：

```
POST_AGE_MAX_MINUTES = 4800 # = 80時間 ≈ 3.3日
```

ただしこれはコーパスの保持期間ではなく、Phoenix の年齢埋め込みテーブルのバケット上限です。つまり「ポスト年齢を特徴量に変換するとき、80時間を超えたらオーバーフローバケット扱いにする」という設計上の境界値です。

コーパス自体の保持期間（いつ追加・削除されるか）を定義するコードはこのリポジトリには存在しません。

Phoenix のコーパス管理は **PhoenixRetrievalClient**（外部サービス）側で行われており、そのサーバー実装はここには含まれていません。

わかっていることをまとめると：

値 根拠

Thunder の保持期間 デフォルト2日 `post\_store.rs` の `Default::default()`

TweetMixer のフィルター `MAX\_POST\_AGE`（非公開） `tweet\_mixer\_source.rs`

Phoenix スコアリングの年齢上限 **80時間（3.3日）** `POST\_AGE\_MAX\_MINUTES = 4800`

Phoenix コーパスの保持期間 **不明**（非公開） このリポジトリに記述なし

▶ よーん

このセッションの会話をそのまま PDF ファイルとしてダウンロードできるようにして

◀ Claude

会話の全履歴が JSONL ファイルに保存されているので、それを読んで PDF に変換します。まず必要なツールを確認します。

◀ Claude

スクリプトを作成してよいですか？ **scripts/** に一時スクリプトを置いてPDFを生成します。
確認が取れたら作成します。

▶ よーん

お願いします

◀ Claude

まず JSONL の構造を確認します。

◀ Claude

構造がわかりました。スクリプトを作成します。

◀ Claude

PDF が生成されました。

- ファイル: **/root/xClaude/outputs/x_algorithm_session.pdf**
- サイズ: 204KB
- ターン数: 352（このセッション全体）

ターミナルから開く場合：

```
xdg-open /root/xClaude/outputs/x_algorithm_session.pdf
# または
evince /root/xClaude/outputs/x_algorithm_session.pdf
```

ご自身のPCにコピーする場合：

```
scp root@<サーバーIP>:/root/xClaude/outputs/x_algorithm_session.pdf ~/Desktop/
```

なお、352ターンはこのセッションファイル全体（note投稿スキルの作業なども含む）の会話数です。Xアルゴリズム解説の部分だけに絞りたいければ、日時や特定キーワードでフィルタリングすることもできます。

▶ よーん

今日のXアルゴリズム についての「<https://github.com/xai-org/x-algorithm.git> をclone して」以降のチャットだけをPDFにして。